

Additional Curated Provider Candidates — Live-Probe Research (2026-06-13)

Research feeding **Defect B** (on-device api-app embeds curated Go providers in `lava-api-go/internal/provider/curated/`). Already shipped: The Pirate Bay (`apibay.org`), YTS (`yts.mx`), Torrents-CSV (`torrents-csv.com`). This document evaluates *additional* candidates against the viability bar.

All probes were live, server-side `curl` GETs/POSTs from a macOS host on 2026-06-13 (~08:55 UTC), bounded to ≤15s each, UA `Mozilla/5.0 (compatible; LavaResearchBot/1.0)`. Per §11.4.6 no-guessing vocabulary: only captured facts are stated; anything not directly probed is marked UNCONFIRMED:.

The bar (ALL must hold, each PROVEN with a captured probe)

1. Free-text query genuinely FILTERS — two distinct queries return DIFFERENT result sets that each contain the query term.
 2. Anonymous — no API key, no login, no cookie.
 3. NOT Cloudflare-challenge-gated for a plain server-side Go GET (no HTTP 403 + `cf-mitigated: challenge` / "Just a moment").
 4. Returns an `info_hash` or magnet.
 5. Stable JSON (preferred) or trivially parseable response.
-

Candidate results

1. Knaben aggregator — VIABLE (with a critical caveat)

- **Search endpoint:** POST `https://api.knaben.org/v1` (`https://api.knaben.eu/v1` issues a 301 → `api.knaben.org`; pin the `.org` host directly to avoid the redirect hop).
- **Live probe:** HTTP 200, `content-type: application/json`. Header `server: cloudflare` + `cf-ray` present, but **`cf-mitigated` ABSENT** and

the body is real JSON — Cloudflare is a passive CDN here, NOT a challenge gate. A plain server-side POST succeeds.

- **Auth:** anonymous; no key/cookie required.
- **info_hash / magnet:** YES — each hit carries both hash (40-hex) and a fully-built magnetUrl (with tracker list).

CAVEAT — the request body shape decides whether the query filters (EZTV-class trap):

The Knaben docs' "rich" body
({"search_type":"score","search_field":"title",
"query":"ubuntu","order_by":"seeders",...}) is a **no-op filter** — score mode ranks the whole corpus by seeders and the query is effectively ignored:

```
Q="ubuntu" search_type=score size=20 → 0/20 titles contain
"ubuntu"
  (returned: "uTorrent Pro 3.6.0 ... Crack", "Internet Download
Manager ...", etc.)
```

The **minimal body** {"query":"<q>","size":<n>} filters correctly. 2-query discrimination proof (minimal body):

```
Q1 {"query":"ubuntu","size":5} → 5/5 titles contain "ubuntu":
  "Ubuntu", "Ubuntu 8.10", "Ubuntu.zip", "Ubuntu Reloaded",
  "Ubuntu installer"
Q2 {"query":"debian","size":5} → 5/5 titles contain "debian":
  "Debian", "Debian DVD", "debian linux", ...
```

Distinct queries → distinct, term-matching result sets. **PASSES the filter bar when the minimal body is used.** Implementation MUST send the minimal body and MUST NOT use search_type:"score".

Captured single-hit JSON shape:

```
{
  "bytes": 732912680,
  "category": "PC",
  "categoryId": [4000000],
  "date": "2009-06-10T22:00:00+00:00",
  "hash": "8E1E7AD6A7198D1BEA2D8564F40EC3480C490301",
  "id": "24bf68e341ce0fbd9259a5d51feed79682ea4eba",
  "magnetUrl": "magnet:?xt=urn:btih:8E1E...&dn=Ubuntu&tr=...",
  "peers": 1,
  "seeders": 0,
  "title": "Ubuntu",
  "tracker": "The Pirate Bay",
  "trackerId": "thepiratebay",
  "virusDetection": 0.3456
}
```

Plus a top-level "total":{"value":809,...}. Knaben is a meta-aggregator — its rows already span TPB, 1337x, etc., so it partly overlaps trackers Lava ships or can't reach directly.

VERDICT: VIABLE. Field mapping: | provider.SearchItem | Knaben JSON key | |---|---| ID / InfoHash | hash (lowercase it) | | Title | title | | MagnetLink | magnetUrl (already built; or rebuild from hash) | | SizeBytes | bytes | | Seeders | seeders | | Leechers | peers – seeders (no separate leechers key) — UNCONFIRMED: whether peers includes seeders; alternative is to set Leechers from peers directly and document | | Date | date (already RFC3339) | | Category | category (string) |

2. BitSearch — VIABLE

- **Search endpoint:** GET <https://bitsearch.eu/api/v1/search?q=<q>&page=<n>> (bitsearch.to issues a 301 → bitsearch.eu; pin .eu directly).
- **Live probe:** HTTP 200, content-type: application/json; charset=utf-8. server: cloudflare + cf-ray present but **cf-mitigated ABSENT** — passive CDN, NOT a challenge gate. Plain server-side GET succeeds.
- **Auth:** anonymous.
- **info_hash / magnet:** YES — infohash (40-hex). No magnet field; build the magnet from infohash + the public-tracker commons (exactly as thepiratebay does in buildMagnet).

2-query discrimination proof:

Q1 q=ubuntu → "ubuntu-19.04-desktop-amd64.iso", "ubuntu-22.04.2-desktop-amd64.iso", "ubuntu-23.04-desktop-amd64.iso"
Q2 q=debian → "debian-12.10.0-amd64-DVD-1.iso", "debian-12.9.0-amd64-netinst.iso", "debian-12.9.0-amd64-DVD-1.iso"

Clean, term-matching, distinct. **PASSES.**

Captured single-result JSON shape:

```
{
  "id": "5cb8afc48700981f3e5b00c4",
  "infohash": "D540FC48EB12F2833163EED6421D449DD8F1CE1F",
  "title": "ubuntu-19.04-desktop-amd64.iso",
  "size": 2097152000,
  "category": 1,
  "subCategory": 7,
  "seeders": 28,
  "leechers": 41,
  "downloads": 0,
  "verified": false,
```

```
"updatedAt": "2026-06-13T08:15:34.896Z"
}
```

Envelope: {"success":true,"query":"ubuntu","results":[...]}.

VERDICT: VIABLE. Cleanest JSON of all candidates. Field mapping: | provider.SearchItem | BitSearch JSON key | |---|---| | ID / InfoHash | infohash (lowercase it) | | Title | title | | MagnetLink | build from infohash + public trackers (no magnet field) | | SizeBytes | size (already bytes, int64) | | Seeders | seeders | | Leechers | leechers | | Date | updatedAt (ISO8601; already RFC3339-compatible) | | Category | category (int — map to string label) |

3. Nyaa.si — VIABLE (XML, not JSON; anime/Asian-media focus)

- **Search endpoint:** GET https://nyaa.si/?page=rss&q=<q>&c=0_0&f=0 (c=0_0 = all categories, f=0 = no filter). RSS 2.0 with a nyaa: XML namespace.
- **Live probe:** HTTP 200, content-type: application/xml, ~71 KB body. No Cloudflare markers, no challenge.
- **Auth:** anonymous.
- **info_hash / .torrent:** YES — each <item> carries <nyaa:infoHash> (40-hex) AND a <link> to <https://nyaa.si/download/<id>.torrent>. Magnet is buildable from the infoHash.

2-query discrimination proof:

Q1 q=naruto → "[Naruto-Kun.Hu] Naruto - 120 [1080p].mkv", ... (all naruto-related)
Q2 q=bleach → "Bleach.S17E01-E26.MULTi.1080p.BluRay.x265-KAF",
"Bleach iTunes (2019) ...", ...

Distinct, term-matching. **PASSES.**

Captured single <item> shape:

```
<item>
  <title>Koha Live CD Release 3 (3.0.4 Ubuntu 9.10 Desktop x86)</
    title>
  <link>https://nyaa.si/download/96659.torrent</link>
  <guid isPermaLink="true">https://nyaa.si/view/96659</guid>
  <pubDate>Tue, 03 Nov 2009 07:03:00 -0000</pubDate>
  <nyaa:seeders>0</nyaa:seeders>
  <nyaa:leechers>1</nyaa:leechers>
  <nyaa:downloads>0</nyaa:downloads>
  <nyaa:infoHash>45008e48c8800b7d7643337b2e70a634e4c69f6a</
    nyaa:infoHash>
  <nyaa:categoryId>6_1</nyaa:categoryId>
```

```
<nyaa:category>Software - Applications</nyaa:category>
<nyaa:size>624.0 MiB</nyaa:size>
</item>
```

VERDICT: VIABLE. XML (encoding/xml in Go), not JSON. <nyaa:size> is a human string ("624.0 MiB") that must be parsed to bytes. Predominantly anime / Asian media — a niche-complementary provider, not a general one. Field mapping: | provider.SearchItem | Nyaa RSS field | |---|---| | ID / InfoHash | <nyaa:infoHash> | | Title | <title> | | MagnetLink | build from <nyaa:infoHash> + public trackers | | SizeBytes | parse <nyaa:size> ("624.0 MiB" → bytes) | | Seeders | <nyaa:seeders> | | Leechers | <nyaa:leechers> | | Date | <pubDate> (RFC1123Z → RFC3339) | | Category | <nyaa:category> |

4. SolidTorrents — REJECTED (duplicate of BitSearch backend)

- GET <https://solidtorrents.to/api/v1/search?...> follows 301 chain solidtorrents.to → bitsearch.to → bitsearch.eu and returns the SAME JSON envelope/shape as BitSearch (`{"success":true,"query":...,"results":[...]}`). location: headers captured confirm the redirect to bitsearch.eu.
 - **VERDICT: REJECTED** — not an independent source; it IS BitSearch. Shipping both would be two providers backed by one upstream (a duplicate, and a §6.E honesty problem). Ship BitSearch only.
-

5. 1337x — REJECTED (Cloudflare challenge-gated)

- GET <https://1337x.to/search/ubuntu/1/> → **HTTP 403** with cf-mitigated: challenge, server: cloudflare, and a "Just a moment..." / _cf_chl_opt JS-challenge body. A plain server-side Go GET cannot pass.
 - **VERDICT: REJECTED** — fails bar #3. (This is exactly why Knaben's *aggregated* 1337x rows are the only honest way to surface 1337x content.)
-

6. TorrentGalaxy — REJECTED (host unreachable)

- torrentgalaxy.to → curl: (6) Could not resolve host. DNS does not resolve from this host.
 - **VERDICT: REJECTED** — no reachable endpoint. UNCONFIRMED: whether a live mirror domain exists; none was probed because the canonical host is dead.
-

7. Torlock — REJECTED (no JSON API; slow HTML)

- GET <https://www.torlock.com/?qq=1&q=ubuntu> → HTTP 200 but content-type: text/html, server: nginx, and the request **timed out at 12s** mid-body (64 KB partial). No JSON endpoint found.
 - **VERDICT: REJECTED** — fails bar #5 (HTML scraping, not trivially parseable, and slow/unreliable). Not worth a curated provider when JSON aggregators already cover the same content.
-

8. torrent-api-py (self-host style) — REJECTED (no stable public host)

- Probed two commonly-cited public deployments: <https://torrentapi-py.vercel.app/api/v1/search?site=1337x&query=ubuntu> → HTTP 404 DEPLOYMENT_NOT_FOUND; and [https://torrent-api-py-nx0x.onrender.com/...](https://torrent-api-py-nx0x.onrender.com/) → HTTP 404 Not Found.
 - **VERDICT: REJECTED** as a *curated upstream* — it is a self-hostable scraper with no stable, project-controlled public host. Even where live, it would just re-scrape Cloudflare-gated sites (1337x) and inherit their fragility. Not an honest fixed-upstream provider.
-

Ranked VIABLE shortlist (ready to implement)

Rank	Provider	Endpoint	Format	Why
1	BitSearch	GET <a href="https://bitsearch.eu/api/v1/search?q=<q>&page=<n>">https://bitsearch.eu/api/v1/search?q=<q>&page=<n>	JSON	Cleanest JSON; size already in bytes; seeders+leechers+infohash all present; no CF challenge; closest structural twin to the shipped thepiratebay/torrentscsv providers — fastest, lowest-risk port.
2	Knaben	POST https://api.knaben.org/v1 body {"query":"<q>","size":<n>}	JSON	Broadest coverage (meta-aggregator across many trackers incl. CF-gated ones we can't reach directly); pre-built magnetUrl. MUST use minimal body — the docs' search_type:"score" body is an EZTV-class no-op filter. Implement the filter-

Rank	Provider	Endpoint	Format	Why
				discrimination test as a hard gate.
3	Nyaa	GET https://nyaa.si/?page=rss&q=<q>&c=0_0&f=0	XML/ RSS	Niche-complementary (anime/Asian media TPB+YTS cover poorly); needs encoding/xml + human-size parsing ("624.0 MiB"). Slightly more parsing work than the JSON pair.

REJECTED: SolidTorrents (= BitSearch backend), 1337x (CF challenge), TorrentGalaxy (DNS dead), Torlock (HTML/slow), torrent-api-py (no stable host).

Single best next-to-implement candidate

BitSearch (bitsearch.eu/api/v1/search). It maps almost one-to-one onto the existing thepiratebay provider shape: a single GET, a flat JSON results[] array, an infohash to build the magnet from (reuse buildMagnet + the publicTrackers commons), size already in bytes, and separate seeders / leechers — no quirks beyond lowercasing the hash and mapping the integer category. No Cloudflare challenge for a plain server-side GET. It is the lowest-effort, lowest-risk addition and should be implemented first, mirroring internal/provider/curated/thepiratebay/ (client.go + provider.go + a live *_realtrackers_test.go whose discrimination assertion runs the two-query filter proof above).

Anti-bluff implementation note (binds whoever ports these): the live test for each new provider **MUST** assert the **2-query filter-difference** captured here (e.g. ubuntu→ubuntu rows, debian→debian rows for BitSearch/Knaben; naruto/bleach for Nyaa), NOT merely "got ≥1 result". For Knaben specifically the test **MUST** also assert that the search_type:"score" body does NOT filter (the recorded no-op), so a future refactor cannot silently reintroduce the EZTV-class bug.